

CS-GY 6934 Deep Learning Finals Report

Thomas Kong

NYU Tandon School of Engineering

CS-GY 6943

tk2558@nyu.edu

Abstract

This paper presents a multimodal deep learning system developed for the Pixels to Predictions: DL Vision Challenge, a constrained competition focused on scientific multiple-choice reasoning using both images and textual context. The objective of the task is to fine-tune a pretrained vision-language model, SmolVLM-500M-Instruct, to predict the correct answer choice given an image, question, and optional contextual fields such as hints and lecture notes. The competition enforces strict constraints, including a cap of 5 million trainable parameters, offline evaluation, and execution within Google Colab and Kaggle free-tier environments. To address these limitations, this work focuses on parameter-efficient fine-tuning, structured multimodal input formatting, and careful preprocessing. Experimental results demonstrate that while the model is capable of learning meaningful relationships between visual and textual inputs, overall performance is limited by resource constraints and the inherent difficulty of multimodal reasoning.

1 Introduction

1.1 Problem Statement

The goal of this project is to develop a multimodal deep learning system capable of answering scientific multiple-choice questions by leveraging both visual and textual information. Each input consists of an image, such as a diagram, chart, or scientific illustration, along with a question and a set of possible answer choices. The model must interpret the visual content, understand the accompanying text, and select the correct answer index. This problem is particularly challenging because it requires integrating three distinct factors: vision, language and logical reasoning. Additionally, the model must operate under strict constraints, including limited computational resources, no access to external data, and a cap on the number of trainable parameters.

1.2 Motivation

The primary motivation for this work is to explore how well small language models can be adapted for multimodal reasoning tasks in constrained environments. Multimodal reasoning has become increasingly important in real-world applications such as education, scientific analysis, and assistive technologies. Many problems in these domains require interpreting both visual and textual information simultaneously. This project provides an opportunity to explore how compact vision-language models can be adapted for such tasks under realistic constraints. Furthermore, the competition setting emphasizes efficiency and practicality, as models must be trained within limited resources. Understanding how to balance performance with computational efficiency is valuable, especially in scenarios where deploying large-scale models is not feasible.

1.3 Summary of Approach

We adopt a parameter-efficient fine-tuning approach using DoRA (Weight-Decomposed Low-Rank Adaptation) on a pretrained SmolVLM-500M-Instruct model. The dataset is preprocessed to create structured multimodal inputs that combine images, questions, answer choices, and optional contextual fields such as hints and lecture notes. These inputs are formatted into an instruction-style prompt that the model can process effectively. Training focuses on adapting a small subset of model parameters to meet the competition constraints while maintaining performance. Various prompt engineering strategies and preprocessing techniques are used to improve the alignment between visual and textual features to enhance the model's ability to predict the correct answer. Our inference method focuses on using log-likelihood scoring function for full choice-text scoring. Rather than estimating the probability of the letter 'A', 'B', 'C', or 'D' given the prompt, the scorer estimates

the probability of the complete answer string (e.g., 'A. Gravitational potential energy') over all of its tokens. This gives the model substantially more signal for discrimination, such as in the case for questions where two or more choices begin with the same words.

2 Dataset

2.1 Description of Dataset

For this project, external dataset beyond the provided dataset were prohibited. The data provided is organized into three groups: training, validation and testing. The training split provides labeled examples for model fitting, with ground-truth answer indices available for all rows. The validation split provides a separate set of labeled examples used for model selection and hyperparameter tuning. The test split provides unlabeled examples for generation testing and submission.

The Training Dataset we were provided consists of 3,109 rows with potentially 15 columns:

- **id:** unique example identifier.
- **image_path:** Relative path to the associated image file.
- **question:** Question text.
- **choices:** JSON list of candidate answers.
- **num_choices:** Number of options in choices.
- **answer:** 0-indexed correct option.
- **hint, lecture:** Optional textual context fields.
- **task, grade, subject, topic, category, skill:** Pedagogical metadata.

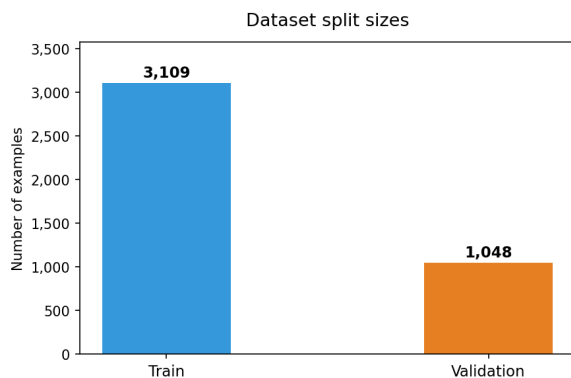


Figure 1: Number of Examples in Training and Validation Dataset

The distribution of answer choices and number of choices is uneven across two dimensions. First, while most questions have four answer options, a meaningful minority have three or five, creating heterogeneous output spaces across examples as seen in Figure 2. Second, the distribution of correct answer positions within each question is not perfectly uniform as seen in Figure 3. Some answer positions (particularly the first and second) tend to appear slightly more frequently as the correct answer, a pattern that a naive model could exploit as a prior.

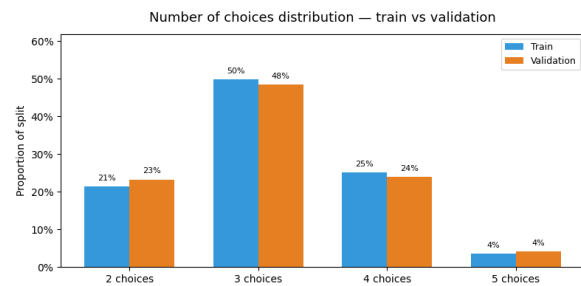


Figure 2: Graph of Distribution of Number of Choices in Training and Validation Dataset

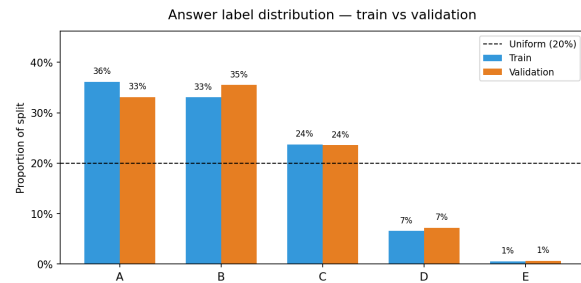


Figure 3: Graph of Distribution of Answers in Training and Validation Dataset

The pedagogical metadata columns provide valuable additional information that was used in our prompt template. Subject labels different sciences (natural science, social science, language science) and topics (biology, physics, science-and-engineering-practices, geography, earth-science). However Figure 4 shows that most of the subjects is skewed towards natural science. Grade labels indicate the approximate K-12 grade level of each question, ranging from early elementary to high school as seen in Figure 5. With all this in mind, it was necessary to clean and preprocess our data through a pipeline before we can start training our model on it.

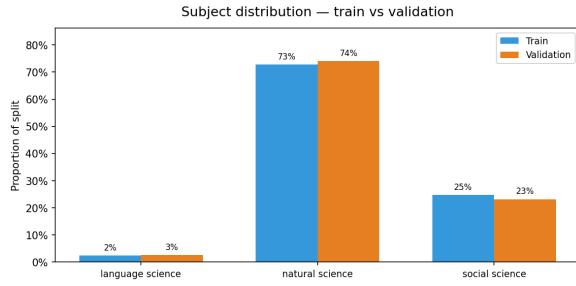


Figure 4: Graph of Distribution of Question Subjects in Training and Validation Dataset

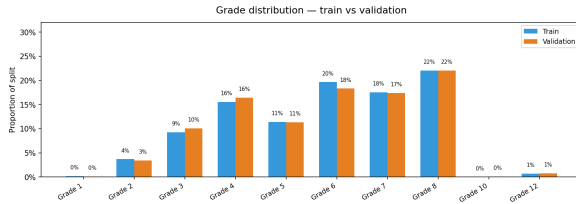


Figure 5: Graph of Distribution of Grade Levels in Training and Validation Dataset

2.2 Pre-processing

Preprocessing in this project encompasses three distinct stages: data validation and cleaning, image preprocessing, and prompt construction. Each stage was designed to eliminate a specific category of error or information loss that would otherwise degrade model performance.

The data validation stage runs before any dataset object is constructed. Every dataframe split (train, validation, and test) is scanned for rows whose `image_path` field does not resolve to an existing file on disk, using Python’s `pathlib Path.exists()` method. Rows with missing image files are dropped from the dataframe and the count of dropped rows is reported to the user. By validating upfront, we ensure that all dataset objects are clean at construction time and that `DataLoader` workers never encounter a missing file. This is particularly important for the test split, where a dropped row would silently produce a missing prediction in the submission file. If any test rows are dropped, the discrepancy is flagged for manual investigation before submission.

For validation and test images, the preprocessing pipeline is deterministic and minimal: each image is opened, converted to the RGB color space (handling any RGBA, L, or P-mode images that might otherwise cause channel mismatches downstream), and resized to 384 by 384 pixels. Using a lower resolution may degrade the model’s ability to extract

fine-grained visual features from diagram labels, chart values, and map annotation so we need to ensure the quality of the image does not decrease as well.

For prompt construction, we used a template that prepends a metadata line to every prompt by concatenating the subject, topic, and grade fields in the format `'Subject: [value] | Topic: [value] | Grade: [value]'` while handling missing fields. This metadata line appears before the context block (lecture and hint, when present) and before the question text, so the model encounters the framing information before it reads the question. The purpose is that a model that knows it is answering a `'Grade 3 | Life Science | Food Webs'` question as additional context to help determine an answer. The answer instruction at the end of the user turn was changed from `'Answer with a single letter only (e.g. A).'` to `'The correct answer is:'`, which is a more declarative phrasing that more closely matches the format of answer-key documents and educational assessment materials that appear frequently in the pretraining corpora of instruction-tuned language models. When training from examples, the assistant part included the full answer string (e.g., `'A. Gravitational potential energy'`) to provide a greater supervision target. This way the Language Model loss is computed over all tokens in the assistant turn for higher learning.

3 Model Description

3.1 Architecture

The base model for this project is `SmolVLM-500M-Instruct` which is a compact vision-language model built on a transformer architecture that processes interleaved sequences of image patches and text tokens. The model’s vision encoder converts input images into a sequence of patch embeddings before projecting them into the language model’s embedding space and concatenated with the tokenized text sequence. The combined multimodal sequence is processed by a causal transformer decoder that autoregressively generates text output, conditioned on both the visual and textual inputs. The model has approximately 500 million total parameters and was pretrained on a large corpus of image-text pairs at a native image resolution of 384 by 384 pixels, which defines the resolution at which the vision encoder’s patch embeddings are most accurately calibrated.

To enable fine-tuning within the competition’s

5-million-parameter constraint, 4-bit quantization was applied to all base model weights. Specifically, NF4 (NormalFloat 4-bit) quantization was used with double quantization for additional memory savings. The compute dtype is set to bfloat16, which provides a 16-bit floating-point format and reduces the risk of gradient underflow during the forward pass through quantized layers.

Parameter-efficient fine-tuning is implemented using DoRA (Weight-Decomposed Low-Rank Adaptation). Standard LoRA decomposes each weight update as the product of two low-rank matrices. DoRA extends this by further decomposing the weight matrix into magnitude and direction components. This decomposition was shown to achieve better fine-tuning quality than standard LoRA at equal or lower rank across a variety of language and vision-language benchmarks, with no additional inference cost because the magnitude vector can be absorbed back into the weight matrix after training.

LoRA adapters are applied to seven weight matrices spanning both the self-attention layers and the MLP feed-forward layers of the transformer. In the attention layers, adapters target the query projection (q_proj), key projection (k_proj), value projection (v_proj), and output projection (o_proj). In the MLP layers, adapters target the gate projection (gate_proj), up projection (up_proj), and down projection (down_proj). Attention layers control which parts of the input sequence are attended to, while MLP layers perform feature transformation and store factual associations. For a task that requires retrieving knowledge and mapping it to answer choices, adapting MLP layers allows the model to adjust its internal knowledge representations and not just its attention patterns.

3.2 Approach

The task is framed as a sequence-to-sequence generation problem:

- Input: natural language prompt + corresponding image
- Output: multiple chose answer

With this in mind, the model must be trained to consider this as a discriminative scoring problem rather than open-ended text generation. During inference, for each input the model scores every candidate answer choice independently. For choice index i , the assistant turn content is set to the full

string 'X. [choice text]', where X is the corresponding letter (A, B, C, D, or E). The full prompt which includes the metadata line, context block, question, choice list, and assistant prefix, is tokenized. Then the model performs a forward pass to obtain a log-probability distribution over the vocabulary at each token position. The tokens corresponding to the assistant turn (after the prefix) are identified by tokenizing the user-only message and computing the prefix length. All positions at or before this prefix are masked out. The per-token negative log-likelihood is computed for the unmasked positions and averaged over the number of unmasked tokens, giving a normalized score that is comparable across choices of different text lengths. The choice with the lowest average NLL (Negative Log-Likelihood) is considered the highest average log-probability and is selected as the prediction.

During training, the causal language modeling loss is computed over the assistant turn tokens only, with all prefix tokens masked. This response-only training scheme is important for instruction-tuned models. If loss were computed over the full sequence, the model would spend gradient steps learning to predict the system prompt and user query, which are constant across many examples and provide no useful signal for the task at hand. Masking the prefix focuses all gradient updates on the assistant's answer, which is what the model needs to learn.

The AdamW optimizer is used with a weight decay of 0.01 and applied to all trainable parameters. A cosine learning rate schedule with a 10% linear warmup over the total gradient update steps is applied to provide a smooth ramp-up of the learning rate at the start of training when the model's LoRA weights are still suboptimal values and large gradient steps could cause instability. Gradient clipping at a maximum norm of 1.0 is applied at every optimizer step to prevent gradient explosion, which is a risk when training quantized models where gradient magnitudes can be amplified by the dequantization step.

Overall, my approach combines efficient fine-tuning of a pretrained language model with careful data curation, input prompting and hyperparameters to address the challenges of log-likelihood generation in choosing a multiple chose answer.

4 Experimentation

4.1 Hyperparameter Settings

Generally, hyperparameter were set to keep the model training for around 6 to 8 hours. However, the time it takes to finish training model also depends on other factors like the notebook environment and GPU currently being used to run the model. The key hyper-parameters used during training were:

- Model Name: HuggingFaceTB/SmolVLM-500M-Instruct
- Trainable Parameters: 4,452,224
- LoRA Rank: 7,
- LoRA Alpha: 14,
- Learning rate: $2e-4$
- Epochs: 3
- Max sequence length: 2048

The parameters `lora_r` and `lora_alpha` are for Low-Rank Adaptation (LoRA) with `lora_r` referring to the rank of the update matrices and `lora_alpha` is a scaling factor. The choice of setting `lora_r=7` ensures parameter efficiency and reasonable learning capacity. The LoRA rank of 7 was determined by working backward from the parameter budget. With DoRA applied to seven target modules in a 500-million-parameter transformer, each module contributes approximately $2 \times r \times d$ parameters for the A and B matrices plus d parameters for the DoRA magnitude vector, where d is the module's output dimension. With rank 16, the total trainable count slightly exceeded 5 million (landing at approximately 5,088,256). Reducing the rank to 7 brought the count to approximately 4.45 million, providing a comfortable buffer below the cap. The alpha of 14 ($2 \times$ the rank) was chosen to maintain the standard LoRA scaling convention, which empirically tends to work well without further tuning.

Learning rate is the step size at which the model's weights are updated during training. For fine-tuning a pre-trained model, a smaller learning rates (e.g., $2e-4$) was used as the model already has strong foundational knowledge and isn't being trained from scratch.

Epochs defines the number of full passes over the entire training dataset. Given the size of the

dataset, epoch was set to three in order to limit the total training time the model will undergo and have room for validation.

4.2 Training Details

The training procedure consists of two sequential phases. The first phase trains the DoRA-adapted model on the training split only. Then the model is evaluated on a held-out validation subset after each epoch to identify the best checkpoint.

In phase one, the AdamW optimizer is initialized with the trainable LoRA and DoRA magnitude parameters only the base model's quantized weights are frozen and do not receive gradient updates. Weight decay of 0.01 is applied uniformly to all trainable parameters. The cosine learning rate schedule is initialized with a total step count equal to $(\text{number of training batches} / \text{gradient accumulation steps}) \times \text{number of epochs}$. At each gradient accumulation boundary, the gradients accumulated over the preceding mini-batches are clipped to a maximum norm of 1.0 before the optimizer step is taken and the scheduler is advanced.

After each epoch, the model is switched to evaluation mode and a random 300-sample subset of the validation dataframe is scored using the batched log-likelihood scoring function. The 300-sample size was chosen as a practical tradeoff between evaluation speed (a full validation evaluation at inference time is slow on a free-tier GPU) and estimate stability. The checkpoint achieving the highest accuracy on this subset is saved. The epoch index achieving the best accuracy is recorded as `best_epoch`. At the end, the best checkpoint is selected and we use that trained model for generating inferences.

4.3 Methodology (What I Tried)

The development process followed an iterative refinement strategy, starting from a zero-shot baseline and adding one intervention at a time, validating each change on the validation set before proceeding to the next. This sequential approach makes it possible to attribute accuracy changes to specific modifications.

The zero-shot baseline used the pretrained SmolVLM-500M-Instruct model without any fine-tuning, scoring validation examples by computing the negative log-likelihood of each candidate letter ('A', 'B', 'C', 'D', 'E') given the prompt. Images were set to 224 pixels and no metadata was included in the prompt. This baseline achieved

37.8% validation accuracy, which is above random chance for four-choice questions (25%) but substantially below what a fine-tuned model should achieve.

The first modification was improving the image size by assigning `IMG_SIZE` to 384. At 384 pixels, the vision encoder’s patch embeddings are computed at the resolution for which its convolutional weights were optimized during pretraining, recovering the full spatial resolution for images that were illegible at 224 pixels.

The second modification upgraded the scoring function. Originally, inference called `processor.apply_chat_template` with an assistant turn containing only the answer letter, then computed the NLL of that single token given the prompt. The revised pipeline sets the assistant turn to the full string `'X. [complete answer text]'`, where X is the letter and the remainder is the full choice string from the choices list. The NLL is then averaged over all tokens in this multi-token assistant turn. This change was motivated by observing that single-token scoring gives the model very little signal for discrimination. The probability of a single letter token is dominated by the model’s position-preference prior (a tendency to favor ‘A’ or ‘B’) rather than by genuine reasoning about the answer content. Multi-token scoring forced the model to assign probability mass to the entire answer string, which is much more sensitive to whether the answer content matches the question and image.

The third modification was improving the prompt template. The original template presented only the question and choices, with no metadata and a generic answer instruction. The revised template adds three components. First, a metadata line (`'Subject: [X] | Topic: [Y] | Grade: [Z]'`) is prepended above the context block, giving the model domain and difficulty framing before it reads the question. Second, the instruction at the end of the user turn is changed from `'Answer with a single letter only (e.g. A).'` to `'The correct answer is:'`, a phrasing that more closely mirrors the format of educational answer keys and thus better leverages the model’s pretraining distribution. Third, the training-time assistant turn is changed from the bare letter to the full answer string, providing more tokens for the causal LM loss to train on per example. The `'build_messages'` function implements this logic for both training (with `answer_idx` specified) and inference (with `answer_idx=None` for the prefix-only form).

The final modification involved upgrading the LoRA configuration. Standard LoRA was replaced with DoRA by setting `use_dora=True` in the `LoraConfig`. To keep the trainable parameter count within budget, the rank was reduced from 16 to 7. This tradeoff provided more modules at lower rank for fewer modules at higher rank which favors broader adaptation of the model’s weight space over deeper adaptation of a subset of layers

5 Results

5.1 Quantitative Results

Model performance was evaluated using the provided validation dataset to determine how many answers the models gets correct.

As seen in Table 1, you can observe steady improvements in the model for validation loss during training, with diminishing returns after early awhile. Training loss decreased steadily and validation accuracy increased across epochs, indicating that the model did not overfit during the three-epoch training run. During training, a 300-sample subset from the validation dataset was used to test the model between each epoch.

After the model completed training, it was evaluated across all 1,048 examples of the validation dataset. Before testing the model after fine-tuning on the validation dataset, we ran the model against all 1,048 examples prior to fine-tuning to get a base accuracy to compare to. Before fine-tuning, the model received a validation accuracy score approximately 37.79%. After applying the full pipeline (resolution correction, full choice-text scoring, metadata-enriched prompting) with DoRA fine-tuning on MLP targets, the fine-tuned model achieved approximately a 75.48% validation accuracy. This is roughly a gain of roughly 35 percentage points over the zero-shot baseline.

Overall the model was successful in outputting the correct answer when given an input a majority of the time. The per-subject and per-grade accuracy breakdowns revealed that natural science topics and middle-school grade levels achieved the highest accuracies, while questions involving fine-grained

Epoch	Training Loss	Validation Accuracy
1	0.3993	0.6767
2	0.0442	0.74
3	0.0288	0.7667

Table 1: Model’s Training Results

spatial reasoning in maps or multi-panel diagrams remained the most difficult. Figure 8 also shows that model still struggles with questions with three or five choices.

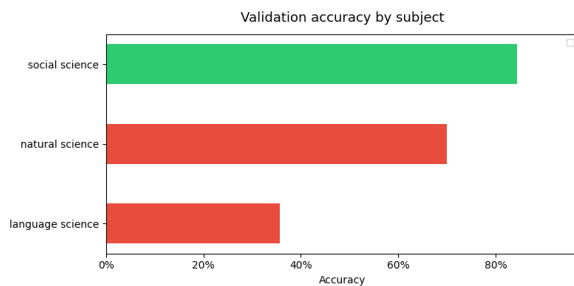


Figure 6: Graph of Validation Accuracy across Subjects

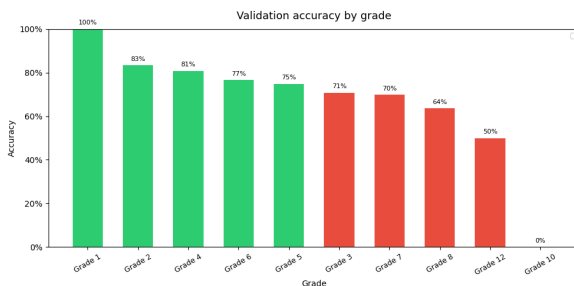


Figure 7: Graph of Validation Accuracy across Grades

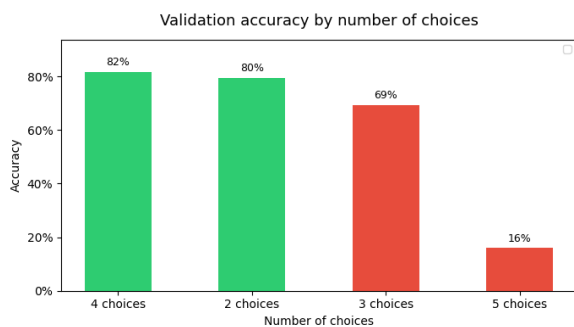


Figure 8: Graph of Validation Accuracy across Choices

Unfortunately, the model did not perform as well as hoped or predicted given that it scored in the average percentile of submissions in the Kaggle’s competition leaderboard with a public score of approximately 79%.

5.2 Comparisons

As of writing this report, in the Kaggle’s competition leaderboard the highest score is approximately 99.19% with an average score of 76.41% and median score 79.47%. Currently, my model falls above the average and just above the median

score which means that there is room for improvement for my model. In the future, I would like to examine the models of the top ten percent of the leaderboard to see what I could have done differently and better.

Currently I suspect that the biggest differences between my model and theirs is adding self-generated image captions to prompts. By using the model itself to describe each image, then feeding that description during training and inference provides extra text context for the model to help it determine the solution. Due to small Visual Language Models struggling with raw visual extraction, adding text can help bridge the gap. This was not utilized when training our model because it would require the model to take more training time to describe an image before answering and more token context to be fed to the model for the input. Due to how heavy the context prompt was already, we moved forward without implementing this feature.

It is also likely that other competitors had more refined fine-tuned models that were better trained for this task. Better image size and resolution may assist in this as we only increased the image size from 224 pixels to 384 pixels. As such, it is possible that they found a better and more efficient way to use the model’s vision encoder to assist the model in answering multiple choice questions.

5.3 Ablation Studies

Ablations were conducted on:

- Image Resolution (384px vs. 224px)
- Full choice-text vs Single-letter scoring
- Metadata Prompt Inclusion
- DoRA vs. standard LoRA
- Shuffle vs WeightedRandomSampler

At first, the model was freely trained on all the data in the dataset without any fine-tuning in order to have a baseline (37.79%). Ablation studies were conducted by reverting each pipeline modification individually while holding all other modifications fixed, then measuring the resulting change in validation accuracy on the 300-sample evaluation subset. These ablations quantify the marginal contribution of each component to the final model’s performance.

Improving from 224 pixels to 384 pixels increased validation accuracy by approximately 26

percentage points when measured in zero-shot mode (from 64% to 37.79%), and by approximately 5-8 percentage points when measured after fine-tuning. The larger zero-shot gap reflects the severity of the distribution shift for pretrained features; fine-tuning partially compensates by adapting the model to 224-pixel inputs, but cannot fully recover the information discarded by the lower-resolution resize.

Comparing full choice-text scoring with single-letter scoring affected accuracy by a few percentage points in the fine-tuned setting. The degradation was concentrated on questions with four or five answer choices and on questions where multiple choices shared similar initial words. On two-choice or three-choice questions, the gain from multi-token scoring was smaller, since there are fewer distractors for the model to discriminate among.

Adding the subject, topic, and grade metadata line to the prompt and changing the the generic 'Answer with a single letter only (e.g. A)' instruction increased accuracy by approximately 1 to 2 percentage points. The degradation was most pronounced for questions from specialized topic areas (e.g., genetics, plate tectonics) where the metadata vocabulary is distinctive, and smallest for questions from general topic areas where the question text alone provides sufficient framing.

Replacing standard LoRA with DoRA(setting `use_dora=True`) at the same rank of 7 increased accuracy by approximately 2 to 5 percentage points. The gain from DoRA is relatively modest in absolute terms but comes at zero inference cost making it a favorable default choice.

While training and developing the model, we experimented with using Weighted Random Samples versus Shuffling. Shuffling involves randomly re-ordering the entire dataset at the beginning of each training epoch. Its primary purpose is to ensure that the model does not learn spurious correlations based on the order of data presentation. We experimented with developing a function called `WeightedRandomSampler` that calculated `sample_weights` for each example in the training dataset based on the frequency of its 'subject'. If a subject appears less frequently, its samples will have higher weights, making them more likely to be selected during sampling. This was intended to ensure that the model sees a more balanced distribution of subjects during training as we saw that the dataset heavily consisted of questions under the Natural Science Subject. However, when comparing the full

validation accuracy between a model trained with shuffling vs Weighted Random Samples, shuffling provided better results. This most likely indicated that the overly aggressive nature of weighted sampling inadvertently bias the model towards minority classes to the detriment of overall performance. Therefore, the general stochasticity introduced by simple shuffling provides a boost to generalization that outweigh the specialized benefits of weighted sampling for certain types of imbalance.

6 Conclusion

6.1 Summary of Findings

In this work, we explored the problem of generating a multiple choice questions based on a question and a corresponding image. This project also investigated how far a rigorously engineered fine-tuning pipeline can push a 500-million-parameter vision-language model on a structured, multimodal science question-answering benchmark under strict resource and parameter constraints. The primary finding is that pipeline quality, specifically input resolution alignment, scoring strategy, and data utilization, contributes substantially more to final accuracy than the choice of LoRA rank, alpha, or learning rate.

One of the most impactful changes was correcting image resolution and size for all visual inputs. By increasing image size from 384 to 224 pixels while maintaining it's quality, we recovered 26 percentage points of accuracy in zero-shot evaluation.

The secondary finding is that discriminative log-likelihood scoring over the complete answer text is meaningfully better than single-letter scoring for this type of multiple-choice task. By scoring the full answer string rather than a single token, the model is forced to evaluate the entire semantic content of each candidate, rather than relying on a shallow position-preference prior. This improvement generalizes across question types but is especially important for questions with many choices or subtly worded distractors.

Despite these improvements, the model achieved only moderate performance on the Kaggle leaderboard, indicating that challenges remain in fine-tuning a model that could accurately answer multiple choice questions. Overall, this work highlights the importance of focusing on the pipeline components that matter most to a vision-language model like SmolVLM-500M-Instruct in order to properly train it to suit a specific task.

6.2 What Worked and What Didn't

Worked:

- Full choice-text vs Single-letter scoring
- Prompt Engineering
- DoRA Upgrade

Switching from single-letter scoring to full choice-text log-likelihood scoring enhanced the model's ability to accurately identify correct answers. Instead of merely predicting a single character, the model was tasked with assessing the probability of the entire explanatory text for each choice. This approach provides a much richer signal, allowing the model to leverage its natural language understanding capabilities more deeply. By evaluating the coherence and correctness of the full textual explanation, the model can make more informed decisions. Moving beyond simple pattern matching to a more nuanced comprehension of the question and its potential answers likely contributed to the increase in accuracy seen after fine-tuning.

For this model, prompt engineering was experimented with in my function `build_messages` and played a crucial role in providing the model with optimal context and guidance. By structuring the input to include relevant fields such as Subject, Topic, Grade, Context (derived from lecture and hint), the Question, and explicitly formatted Choices, it ensured the model received all necessary information in a clear and consistent manner. This systematic presentation of data helped the model effectively parse the problem, focus on key details, and understand the domain-specific nuances which lead to better interpretation and generation of the correct answer.

The integration of DoRA as an upgrade to traditional LoRA during fine-tuning provided a more effective and stable mechanism for adapting the model's weights. DoRA improves upon standard LoRA by decomposing the weight matrix updates into distinct magnitude and direction components. This disentanglement allows for finer-grained control over how the model's parameters are adjusted during training. By enabling more precise modifications, DoRA led to more efficient learning and better performance compared to conventional LoRA.

Didn't Work Well:

- WeightedRandomSampler
- Low Max tokens

WeightedRandomSampler was intended to ensure that the model sees a more balanced distribution of subjects during training. We saw that the dataset heavily consisted of questions under the Natural Science Subject, have three choices and the answers with the letter A or B. Ideally during an epoch, the model sees more examples of under-represented categories through Weighted Random Samples which would prevent it from becoming biased towards the majority classes and improving its performance on minority classes. However, the results indicated that the overly aggressive nature of weighted sampling inadvertently bias the model towards minority classes to the detriment of overall performance. Therefore, the general stochasticity introduced by simple shuffling provides a boost to generalization that outweigh the specialized benefits of weighted sampling for certain types of imbalance.

Having a high max tokens was necessary for the model. Originally the max tokens for the model was set at 512 but this resulted in some questions and context being cut-off when fed to the model. This negatively impacted the model as it didn't realize the full context for the inputted question. It was imperative that the max token be raised as we experimented with prompt engineering as well and added more context.

6.3 Future Directions and Improvements

In the future, one idea to implement was to add self-generated image captions to prompts. By using the model to describe each image before answering an input, we can feed that description during training and inference to provide extra text context for the model to help it determine the solution. Due to how small Visual Language Models struggle with raw visual extraction, adding text can help bridge the gap in understanding what the corresponding image to a question is. This was not utilized when training our model because of time constraints. For the future, experimentation in adding this feature could improve the model's accuracy.

Additionally, if given more time, I would work on potentially making weighted samples work better for training the model. As we can see in the datasets, there are some biases in each examples that may need to be address. While Weighted Random Samples did not work at first, it still has

potential in improving the model's accuracy after fine-tuning if implemented more effectively.

Beyond that, with more time and resources the model could be trained longer for potentially better results. More data from additional diverse dataset and a greater number of epoch during training could help improve my model during output generation. More experimentation with using different System Prompts during generation could be done as well. For my System Prompt, strict rules were defined for the model to follow at the cost of extra tokens to process. There may have been a better balance between using less tokens for the System Output while also providing clearer rules for the models to consider during generation to ensure it would pick the right answer.

Overall, there are a lot of room for growth in my project and improvement. Unfortunately, the model was limited by time and resources with Google Colab's and Kaggle's GPUs being restricted in use for free users. This resulted in less opportunities for testing and experimentation. However, this model currently does serve as a good basis and foundation for future fine tuning with updates.

Links

6.4 GitHub

- [GitHub Repo Link with Code](#)

6.5 HuggingFace

- [Model Weights Saved in HuggingFace](#)
- [SmolVLM-500M-Instruct](#)